

Name: _____ Number: _____

Part 1: Programming

Consider the following class.

```
import random

class Character:
    def __init__(self, name, era_type, hp, atk):
        self.name = name
        self.era = era_type
        self.hp = hp
        self.atk = atk
        self.defending = False

    def attack(self, other):
        damage = self.atk
        other.hp -= damage
        print(f"{self.name} hits {other.name}! {other.name} loses {damage} HP.")

    def defend(self):
        self.defending = True
        print(f"{self.name} takes a defensive stance.")

    def is_alive(self):
        return self.hp > 0

class Arena:
    def __init__(self):
        self.fighters = []

    def add(self, character):
        self.fighters.append(character)

    def fight(self):
        alive = [c for c in self.fighters if c.is_alive()]
        print(" Arena Battle Begins!\n")
        while len(alive) > 1:
            for c in alive:
                targets = [e for e in alive if e != c]
                if not targets:
                    break
                target = random.choice(targets)
                action = random.choice([c.attack, c.defend])
                if action == c.attack:
                    action(target)
                    print(f"{target.name} now has {max(0, target.hp)} HP remaining.\n")
                else:
```

Name: _____ Number: _____

```
        action()
        alive = [c for c in self.fighters if c.is_alive()]
    if len(alive) == 1:
        winner = alive[0]
        print(f" {winner.name} from {winner.era} wins the battle!")
    else:
        print(" The battle ended with no clear winner-")

ambatus = Character("Ambatus", "Lusitanian, II BC", 100, 15)
caius = Character("Caius", "Roman, I AD", 120, 12)
fredrick = Character("Fredrick", "Goth, V AD", 110, 13)
ali = Character("Ali", "Moorish, IX AD", 100, 14)
sancho = Character("Sancho", "Portuguese Templar, XII AD", 130, 11)
gil = Character("Gil", " Portuguese Sailor, XV AD", 105, 16)

arena = Arena()
for fighter in [ambatus, caius, fredrick, ali, sancho, gil]:
    arena.add(fighter)

arena.fight()
```

1.1. Create a new character called "Maria". She has a Health Points of 120 and an Attack Power of 16. "Portuguese revolutionary"

1.2. The result may be that "The battle ended with no clear winner" in the fight method. This happens because of mutual elimination or all being defeated. How to avoid this situation?

Name:_____ Number:_____

1.3. What is the purpose of the following code?

```
class Warrior(Character):
    def __init__(self, name, era_type):
        super().__init__(name, era_type, hp=120, atk=20)

    def attack(self, other):
        print(f"{self.name} charges fiercely!")
        super().attack(other)
```

1.4. Create a new character called "Portuguese revolutionary". This Character is called "Maria". She has 120 Health Points and 16 Attack Power. How can we do it by creating a subclass of Character? Ignore the class arena.

Name: _____ Number: _____

Part 2: Machine Learning

```
# =====
# Comment 1:
# =====

import nltk
import spacy
import numpy as np
import matplotlib.pyplot as plt
import requests
import textstat
from textblob import TextBlob
from nltk.sentiment.vader import SentimentIntensityAnalyzer
nltk.download("punkt", quiet=True)
nltk.download("vader_lexicon", quiet=True)
NLP = spacy.load("en_core_web_sm")
# =====
# Comment 2:
# =====

class TextLoader:
    @staticmethod
    def from_url(url):
        return requests.get(url, timeout=10).text
# =====
# Comment 3:
# =====

class DiplomaticSpeechAnalyzer:
    def __init__(self, text):
        self.text = text
        self.words = nltk.word_tokenize(text)
        self.doc = NLP(text)
        self.vader = SentimentIntensityAnalyzer()
        self._compute()
    def _compute(self):
        blob = TextBlob(self.text)
        self.textblob = {
            "polarity": float(blob.sentiment.polarity),
            "subjectivity": float(blob.sentiment.subjectivity)
        }
        self.vader_sentiment = self.vader.polarity_scores(self.text)
        self.features = self._features()
        self.politeness = self._politeness()
    def _features(self):
        t = self.text.lower()
        return {
            "gratitude": self._count(t, ["thank", "grateful", "appreciate"]),
```

Name: _____ Number: _____

```
        "deference": self._count(t, ["honor", "privilege", "pleased"]),
        "mitigation": self._count(t, ["would like", "wish to", "may i"]),
        "cooperation": self._count(t, ["cooperation", "dialogue", "partnership"]),
        "hedging": self._count(t, ["perhaps", "maybe", "seems", "appears"]),
        "conflict": self._count(t, ["war", "threat", "crisis", "risk"]),
        "directness": sum(1 for tok in self.doc if tok.text.lower() == "you"),
        "readability": float(textstat.flesch_reading_ease(self.text))
    }

def _count(self, text, words):
    return float(sum(text.count(w) for w in words))

def _politeness(self):
    f = self.features
    scale = max(len(self.words), 1) / 1000
    return {
        "gratitude": f["gratitude"] / scale,
        "deference": f["deference"] / scale,
        "mitigation": f["mitigation"] / scale,
        "cooperation": f["cooperation"] / scale
    }

def politeness_score(self):
    p = self.politeness
    return float(
        0.3 * p["gratitude"] +
        0.2 * p["deference"] +
        0.3 * p["mitigation"] +
        0.2 * p["cooperation"]
    )

def vector(self):
    return {
        "textblob_polarity": self.textblob["polarity"],
        "textblob_subjectivity": self.textblob["subjectivity"],
        "vader_compound": self.vader_sentiment["compound"],
        "politeness_score": self.politeness_score(),
        "cooperation": self.features["cooperation"],
        "conflict": self.features["conflict"],
        "hedging": self.features["hedging"],
        "directness": self.features["directness"]
    }

# =====
# Comment 4:
# =====

class Visualizer:
    def compare(self, a1, a2, name1="A", name2="B"):
        v1 = a1.vector()
        v2 = a2.vector()
        metrics = list(v1.keys())
        x = np.arange(len(metrics))
```

Name: _____ Number: _____

```
w = 0.35
plt.figure(figsize=(10, 5))
plt.bar(x - w/2, list(v1.values()), width=w, label=name1)
plt.bar(x + w/2, list(v2.values()), width=w, label=name2)
plt.xticks(x, metrics, rotation=45)
plt.title("Diplomatic Feature Comparison")
plt.legend()
plt.grid(True, axis="y")
plt.tight_layout()
plt.show()

def sentiment_trend(self, analyzer, title):
    sentences = nltk.sent_tokenize(analyzer.text)
    scores = [TextBlob(s).sentiment.polarity for s in sentences]
    plt.figure(figsize=(10,4))
    plt.plot(scores, marker="o")
    plt.title(title)
    plt.xlabel("Sentence")
    plt.ylabel("Polarity")
    plt.grid(True)
    plt.show()

# =====
# Comment 5:
# =====

class Report:
    def show(self, a, name):
        v = a.vector()
        print(f"\n===== {name} =====")
        print("TextBlob:", a.textblob)
        print("VADER:", a.vader_sentiment)
        print("Politeness Score:", round(v["politeness_score"], 3))
        print("Cooperation:", v["cooperation"])
        print("Conflict:", v["conflict"])
        print("Hedging:", v["hedging"])
        print("Directness:", v["directness"])

# =====
# Comment 6:
# =====

if __name__ == "__main__":
    URL_TRUMP =
"https://raw.githubusercontent.com/masterfloss/text/refs/heads/main/Trump260428.txt"
    URL_CHARLES =
"https://raw.githubusercontent.com/masterfloss/text/refs/heads/main/KingCharlesIII260429.txt"

    trump = DiplomaticSpeechAnalyzer(TextLoader.from_url(URL_TRUMP))
    charles = DiplomaticSpeechAnalyzer(TextLoader.from_url(URL_CHARLES))
    report = Report()
    viz = Visualizer()
```

Name: _____ Number: _____

```
report.show(trump, "Trump")
report.show(charles, "Charles")
viz.compare(trump, charles, "Trump", "Charles")
viz.sentiment_trend(trump, "Trump Sentiment Trajectory")
viz.sentiment_trend(charles, "Charles Sentiment Trajectory")
```

2.1. What is the purpose of the code following the comments:

Comment 1:

Comment 2:

Comment 3:

2.2. Describe the purpose of the code presented above.

Comment 4

Comment 5:

Comment 6:

2.3. What kind of algorithms were used previously?

Name: _____ Number: _____

Part 3: Data Science

1. What are the components of POST-DS, and is it integrated with CRISP-DM?
2. During the classes, was the POST-DS approach presented? How may it be used to support a project?
3. What is Machine Learning?
4. What is Artificial Intelligence?